

🔍 <https://javascript.com/>



🔍 javascript

JavaScript

李建輝 Gary

garylee@ispan.com.tw

課程大綱

- 1. JavaScript 簡介與基本語法
- 2. 變數與資料型態
- 3. 運算子(Operator)
- 4. 流程控制
- 5. JavaScript物件
- 6. 函數(Function)
- 7. 內建物件方法介紹
- 8. HTML DOM物件
- 9. 事件處理 1
- 10. 事件處理 2
- 11. BOM物件
- 12. JSON介紹 & AJAX

🔍 <https://javascript.com/>



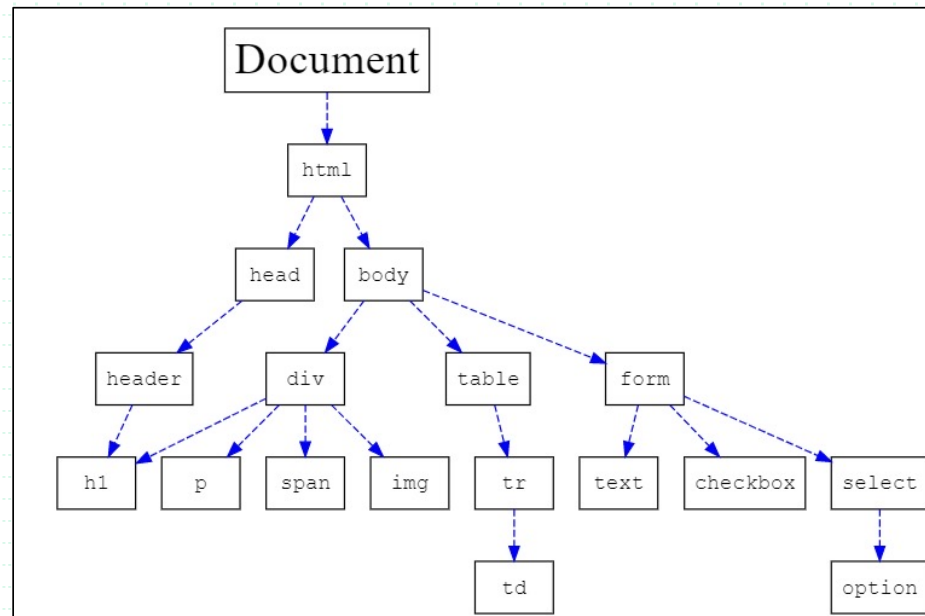
1. JavaScript簡介與基本語法

JavaScript 簡介

- JavaScript是網景公司(Netscape co.) 公司開發的一種腳本語言
 - 1995年公布第一個版本
 - 開發初期稱為LiveScript，之後搭上Java名氣改為JavaScript
- 1997年統一標準前端語法ECMA-262簡稱ECMAScript標準
 - <https://www.ecma-international.org/ecma-262/>
- JavaScript 包括的三大部分
 - ECMAScript
 - DOM(Document Object Model)
 - BOM(Browser Object Model)

DOM (Document Object Model)

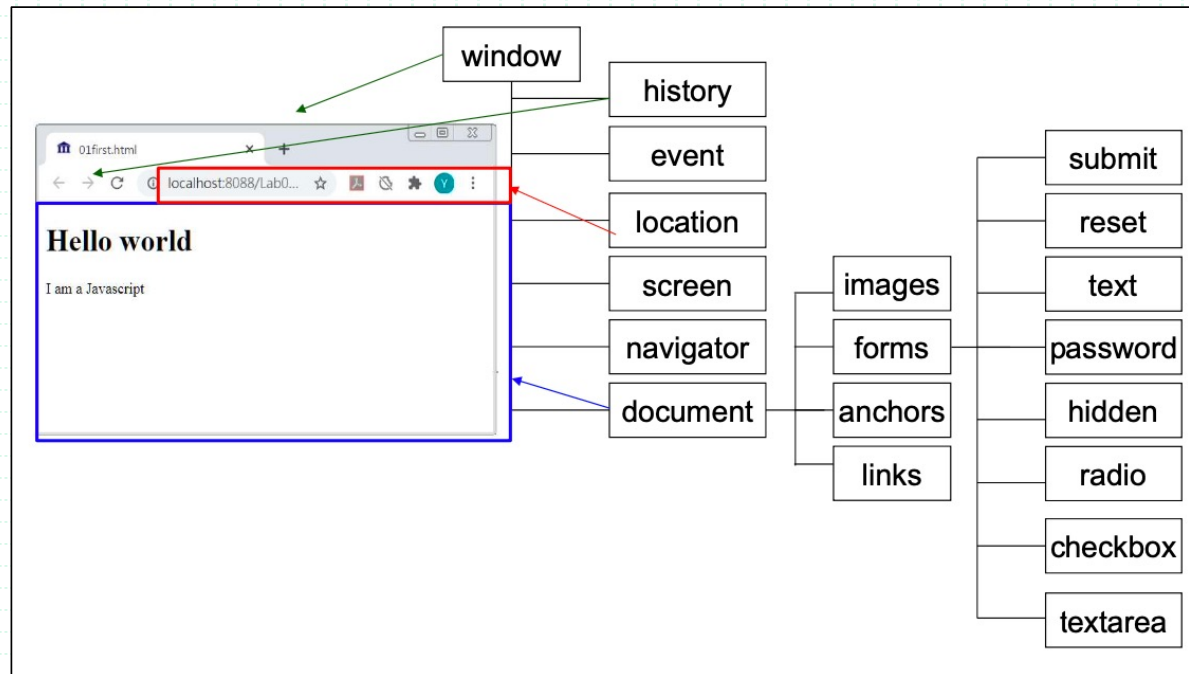
- DOM是一個以樹狀結構表示HTML文件模型，並定義讓程式可以存取及改變文件架構樣式內容方法



https://javascript.com/

BOM (Browser Object Model)

- BOM是瀏覽器物件模型，是瀏覽器所有功能的核心



JavaScript 簡介

- 2005年以Ajax為契機回歸主流，進入HTML5
 - Asynchronus JavaScript And XML (XMLHttpRequest API)
- JavaScript大改變2015年發布ES6版本
 - 新一代的JavaScript語法
 - ECMAScript語法標準化組織-目前ES13(2022)

JavaScript 特性

- JavaScript是一種腳本語言
 - 是一種鬆散(loosely)式的程式語言
- 直譯式語言
 - 執行程式碼時解析、轉譯、執行同時處理
- 由多種功能構成
 - 網頁動態效果、進行表單驗證、事件偵測監聽處理、存取cookie等
- 可應用(衍生)於各種環境
 - 最早是以瀏覽器為執行環境設計的程式語言，但現在已不再局限於此，例如以伺服器環境為主的Node.js、以強型別物件導向的TypeScript

前端分類分工

- HTML 網頁架構

- 骨頭

- CSS 外觀樣式 排版

- 皮膚

- JavaScript 程式邏輯

- 大腦

HTML the Skeleton



CSS the Skin



Javascript the Brain



前端越來越複雜

● 前端分工詳細

➤ 前端設計

- ✓ UI 使用者介面與設計
- ✓ UX 使用者互動、使用者體驗
- ✓ RWD響應式設計

➤ 前端功能

- ✓ SPA(單頁式應用)
- ✓ SSR(伺服器渲染)
- ✓ CMS(內容管理)

● 前端後端分離

- 前端負責控制頁面與資料渲染
- 後端負責商業邏輯與數據資料
- 經由AJAX向RESTful API接口發送請求獲取JSON資料
- 減少後端Server流量乘載，頁面渲染更快，使用者體驗更佳

TypeScript

- JavaScript 一開始設計只是為了控制 DOM，但隨著 AJAX 與 SPA、SSR 出現後，為了更好的使用者體驗，前端工程師開始將部分商業邏輯寫在 JavaScript，這也使得前端出現上千行的程式碼，JavaScript 的**可維護性**一直為人詬病受到質疑，因此我們需要能寫**複雜商業邏輯**的語言
- 微軟開發的**強型別**的JavaScript

前端框架Framework

● 前端框架

- 框架是個針對軟體構建提供一個基本模板，可以幫助軟體工程師有效的開發專案並維護程式。
- 元件 Components
- 狀態管理State Management
 - ✓ 管理資料和事件流程
- 生命週期Life Cycle
 - ✓ 渲染機制
- 路由Routing (Client-Side)
 - ✓ 前端頁面之間導航切換

● 目前前端框架

- Angular
 - ✓ Google推出
- React
 - ✓ Meta(前身Facebook)推出
- Vue
 - ✓ 尤雨溪(前google工程師)團隊推出
- Svelte
 - ✓ 2019年由Rich Harris推出
- 還有很多前端框架選擇

<script> 標籤

- `<script type="text/javascript">` 程式撰寫與此 `</script>`
 - 腳本區塊
 - 程式撰寫與此
- 標籤位置
 - `<body>` 任意位置 – 不建議
 - `<head>` 標籤內 – 特定事件會撰寫於此
 - `<body>` 標籤內 `</body>` 結束標籤前
- 外部讀取方式
 - `<script src="main.js"></script>`

🔍 <https://javascript.com/>



JavaScript使用

- 顯示對話方塊及按鈕

```
alert("message");
```

- 在除錯模式(F12, Ctrl+Shift+I)控制台輸出內容

```
console.log(exp);
```

- HTML頁面body輸出內容

```
document.write(exp);
```



JavaScript使用

- 單行註解

- win:ctrl + /
- mac:command + /

// 單行註解

- 雙行註解

- win:shift + alt + A
- mac:option + shift + A

/*
多行註解
*/

JavaScript 撰寫規則

- 以分號 (;) 結束
 - 結尾沒分號不會發生錯誤，但不建議不寫
- 陳述句可含空白、換行或Tab
 - 換行必須是在有意義的關鍵字或符號之後，不然會造成錯誤
- 要區分大小寫
- 多個陳述句不建議寫在同一行
 - 一行一個陳述句
- 註解comment
 - // 單行註解
 - /* 多行註解 */

🔍 <https://javascript.com/>



2. 變數與資料型態

變數宣告

- JavaScript是一種鬆散(loosely)程式語言
 - 嚴格來說變數宣告在JavaScript非必要，但不建議不宣告
 - 一般變數宣告後不分資料型態，真正的資料型態將由指定的敘述定義來決定
- 宣告變數
 - `var 變數名稱 [= 初始值];`
 - 可重複宣告
 - 函數範圍

變數命名規則

- camelCase法
 - 開頭單字第1個字小寫，之後單字第1個字元都大寫
 - studentName、windowWidth
- 變數必須使用字母、下底線、錢號作為開頭
 - 不能數字開頭
- 變數名稱不可是保留字
- 命名是讓程式碼易於閱讀
 - 建議不要縮寫，不易於了解資料內容(psw、usr、w、acc、adr)
 - 垃圾程式碼書寫準則(反話書寫，反者做才是正確)

保留字

●ECMAScript 2015 保留關鍵字

var	let	if	for	else
const	break	do	switch	case
continue	import	export	in	true
false	finally	function	instanceof	new
this	try	catch	typeof	while

https://www.w3schools.com/js/js_reserved.asp

變數宣告

- 另一種宣告變數
 - `let` 變數名稱 [= 初始值];
 - ES6 新增指令
 - 不允許變數重複
 - 區塊範圍
 - 建議用此方式宣告

常數宣告

● 宣告常數

- `const` 常數名稱 [= 初始值];
- ES6 新增指令
- 設定後不能改變只能設定一次
- 通常會全部使用大寫字母，單字間以底線區隔
 - ◆ `PI`、`CONSUMPTION_TAX`、`SQL_UPDATE`

資料型態

● 基本型

- 可以設定內容
 - ✓ string：字串
 - ✓ number：數值
 - ✓ boolean：布林值
- 不可設定內容
 - ✓ null：空物件
 - ✓ undefined：未定義的值

● 參考型

- array：陣列
 - ✓ 資料集合
 - ✓ 可透過索引值存取標籤
- object：物件
 - ✓ 資料集合
 - ✓ 可透過名稱存取標籤
- function：函數

資料型態

● string 字串

- 單引號 (')
- 雙引號 (' ')
- 模板字串 反引號 (`)
 - ✓ ES6新增
 - ✓ 字串嵌入變數
 - ◆ 使用 \${xxx} 格式
 - ✓ 支援多行字串
- 跳脫字元 \ + 字串

字串	說明
\b	退格鍵
\\	反斜線
\'	單引號
\"	雙引號
\f	換頁字元
\n	換行字元

資料型態

● number 數值

➤ 整數型別

- ✓ 支援到15位
- ✓ 2進制須以0b開頭 0b11
- ✓ 8進制須以0o開頭 0o600
- ✓ 16進制須以0x開頭 0xFFffFF

➤ 浮點數型別

- ✓ 3.14
- ✓ 指數方式表達 3.13e5

● boolean 布林值

- 若在真假判斷式中，任何值都可轉為布林值。0、NaN、""、null、undefined 轉為false
- 其它的值，包括所有物件與陣列則轉為true

資料型態

- undefined 未定義值

- 變數已宣告但未給值
- 參照未定義的屬性
- 函數沒有回傳值
- 回傳undefined類別

- null 空值

- 空物件
- 回傳的object類別

- 偶爾會被搞混 NaN

- number資料型態
- Not a Number

判斷資料型態

- 取得變數的基本資料類型
 - 使用typeof 語法

```
console.log(typeof 'Gary')  
console.log(typeof 3.14)  
console.log(typeof true)  
console.log(typeof null)  
console.log(typeof [1,2,3,4])
```

🔍 <https://javascript.com/>



3. 運算子

運算式簡介

- 運算式為運算子和運算元組合而成

$$5 = 3 + 2$$

二元運算式 = 運算元 + 運算子 + 運算元

➤ 算數運算子

✓ + - * / % ++ --

➤ 賦值(指派)運算子

✓ = += -= *= /= %=

➤ 比較(關係)運算子

✓ == != < > <= >=

➤ 邏輯運算子

✓ && || !

運算式簡介

● 運算式類型

➤ 運算邏輯分類

- ✓ 算術運算子
- ✓ 賦值(指派)運算子
- ✓ 比較(關係)運算子
- ✓ 邏輯運算子
- ✓ 條件運算子

➤ 運算元分類

- ✓ 一元運算子
- ✓ 二元運算子
- ✓ 三元運算子

算術運算子

- 算術運算子包含標準四則運算，主要功能執行基本數學計算

運算子Operator	說明	範例
+	加法	1+2=3
-	減法	10-1=9
*	乘法	2*2=4
/	除法	10/2=5
%	取餘數	10%3=1
++	遞增	a++;
--	遞減	a--;

邏輯運算子

● AND運算子 &&

A	B	A && B
true	true	true
true	false	false
false	false	false
false	true	false

● OR運算子 ||

A	B	A B
true	true	true
true	false	true
false	false	false
false	true	true

● NOT運算子 !

A	!A
true	false
false	true

賦值(指派)運算子

運算子Operator	說明	範例
=	可以將右側值賦值給左側變數	<code>a = 5</code>
+=	等於 <code>a = a + 5</code>	<code>a += 5</code>
-=	等於 <code>a = a - 5</code>	<code>a -= 5</code>
*=	等於 <code>a = a * 5</code>	<code>a *= 5</code>
/=	等於 <code>a = a / 5</code>	<code>a /= 5</code>
%=	等於 <code>a = a % 5</code>	<code>a %= 5</code>

比較(關係)運算子

運算子Operator	說明	範例
==	比較是否相等(值)	10 == '10' true
===	嚴格比較是否相等 (值及資料型別)	10 === '10' false
!=	比較是否不相等	10 != '10' false
!==	嚴格比較是否不相 等	10 !== '10' true
>	大於	10 > 10 fasle
<	小於	10 < 10 false
>=	大於等於	10 >= 10 true
<=	小於等於	10 <= 10 true

條件運算子

- 會依據條件式的真偽輸出相對應的語句
 - 條件為true時，執行語句1
 - 條件為false時，執行語句2

條件式 ? 語句1 : 語句2

優先順序度

- 建議使用括號明確表示優先順序

優先順序	運算子Operator
高	陣列 [] 括弧 ()
	遞增 ++ 遞減 -- 否定 !
	乘法 * 除法 / 餘數 %
	加法 + 減法 - 字串連接 "" + ""
	位移 >> <<
	比較 < <= == >
	等於 == 不等於 !=
	嚴格等於 === 嚴格不等於 !==
	AND & XOR ^ OR
	邏輯積 && 邏輯和
	條件 true ? x : y
	指派 = += -=
低	逗號 ,

🔍 <https://javascript.com/>



4. 流程控制

流程控制

- 流程控制可分3類
 - 循序結構
 - ✓ 撰寫順序執行
 - 選擇結構
 - ✓ 依條件判斷選擇執行
 - 重複結構
 - ✓ 重複執行特定處理
- 使用以上流程控制稱為**結構化程式設計**

依條件判斷處理- if指令

- JavaScript提供if 及 switch 可依條件不同 執行不同程式
- if 指令
 - 意思為『若...就...，否則就...』
 - 依條件為true or false 執行不同區塊

```
if(條件式){  
    條件式為true時執行此指令區塊  
}else{  
    條件式為false時執行此指令區塊  
}
```

依條件判斷處理- if指令

- else if 多條件分支

```
if (條件式1){  
    條件式1為true時執行此指令區塊  
}else if(條件式2){  
    條件式2為true時執行此指令區塊  
}else{  
    所有條件皆為false時執行此指令區塊  
}
```


依條件判斷處理- switch指令

● switch指令

1. 計算最前面條件式
2. 執行比較結果為真的case區塊
3. 若沒有符合的case,則呼叫最後的default

● 注意

- 不會自動終結執行switch指令
 - ✓ break 語句才會跳出區塊
- case值以===運算子比較

依條件判斷處理- switch指令

● switch指令

```
switch (條件式){  
    case 1:  
        符合條件式=值1時要執行指令  
        break;  
    case 2:  
        符合條件式=值2時要執行指令  
        break;  
    default:  
        條件式不符合上列所有值時要執行的指令  
        break;  
}
```

重複結構控制迴圈- while指令

●while指令

- 前置判斷視條件是否成立，若成立反覆執行或迴圈內程式可能1次都沒有執行

```
while(條件式){  
    條件式為true時執行的程式  
}
```

重複結構控制迴圈- do...while指令

● do...while指令

- 後置判斷，不論條件是否成立，迴圈內程式至少會執行1次

```
do{  
    條件式為true時執行的程式  
} while(條件式);
```

- 兩者的執行結果差異會發生在迴圈開始前條件式為false時
- 無窮迴圈

重複結構控制迴圈- for指令

- for指令

- 可自行設計控制迴圈的處理次數

```
for( 初始化運算式; 迴圈繼續條件式; 遞增/遞減式){  
    迴圈內執行指令區塊  
}
```

依序列列舉陣列- for... of指令

● for... of指令

- ES6新增語法
- 依序列列舉陣列
- 類陣列 NodeList arguments iterator

```
for(自定義變數 of 可列舉物件){  
    迴圈內執行指令區塊  
}
```

🔍 <https://javascript.com/>



5. JavaScript物件

何謂物件

- 物件表示一個持有多個屬性及方法資訊的東西
- 在JavaScript中物件是可透過名稱存取的關聯陣列

```
{屬性名稱: 值, 屬性名稱: 值, ...};
```



JavaScript物件種類

- JavaScript內建物件(Built-in Object)
 - JavaScript定義的標準物件
 - Math、String、Number ...
- 瀏覽器物件BOM
 - JavaScript須在特定環境(瀏覽器)中才可以使用
- DOM物件
 - 操作HTML/XML的物件群組
- 自定義物件
 - 由自己創建的物件

JavaScript物件

- 物件 = 屬性 + 方法

//新增物件第一種方式

```
var obj = new Object();
```

//新增物件第二種方式

```
var obj2 = {};
```

🔍 <https://javascript.com/>



6. 函數 Function

何謂函數

- 函數為基於輸入值(參數)執行某些處理後，將其結果回傳的機制
- 四種自定義函數的方法
 - 利用function指令
 - 利用Function物件建構子-**不建議**
 - 利用函數常值
 - 利用箭頭函數
 - ✓ ES6新增

利用function指令

●function指令

```
function 函數名稱(參數,...){  
    ...任意處理程序...  
    return 傳回值;  
}
```

利用函數常值定義

- 此函數宣告時又稱為匿名函數

```
var 變數名稱 = function(參數,...){  
    ...任意處理程序...  
    return 傳回值;  
}
```

利用Function物件建構子定義

- Function() 建構子定義
 - 不建議 此方式建構函數

```
var 變數名稱 = new Function(參數,...,函數本體);
```

利用箭頭函數(Arrow function)定義

● 箭頭函數

- ES6新增
- 函數本體處理後的值後即傳回，省略return
- 有**固定this**指向全域或所創建的物件

```
var 變數名稱 =( 參數,... ) => { ...函數本體... }
```


function 參數預設值

- 參數預設值
 - ES6新增
 - 參數 = 預設值

```
function 函數名稱( 參數=值,...) {  
    ...任意處理程序...  
    return 傳回值;  
}
```

作用範圍比較

- 變數在整個程式中參考的為全域範圍 = 全域變數
 - ✓ 在函數外宣告的變數
- 變數只能在定義函數中參考的區域範圍 = 區域變數
 - ✓ 在函數內宣告的變數

this關鍵字

- 會根據建構子的上下文中，**this**表示建構子建立的實體
 - ✓ **this**指向的是一個物件(變數)，這個物件會依照上下文物件改變

位置	this的參考目標
最高層級(函數外)	全域物件 window
函數	全域物件
call / apply 方法	參數指定物件
事件監聽 EventListener	事件觸發者
建構子 Constructor	建立的實體
方法 Method	呼叫的物件

原型Prototype物件

- 在JavaScript物件導向裡是『原型』不是『類別』
- 在JavaScript世界中沒有嚴謹類別，而是使用函數創建建構子
函數原型物件類別

```
function MyClass(參數,...){  
    this.屬性名稱=值;  
    this.方法名稱= function(){函數本體}  
}
```

- 功能在初始化將建立的物件，不需要回傳值

🔍 <https://javascript.com/>



7. 內建物件方法介紹

JavaScript 內建物件

- 內建物件(Built-in Object)是指JavaScript中標準內建，在所有JavaScript環境皆可使用，相對瀏覽器物件需要在瀏覽器中才可以使用
- 提供基本功能物件如下
 - Array、String、Date、Math、RegExp、Function、Number...等

🔍 <https://javascript.com/>

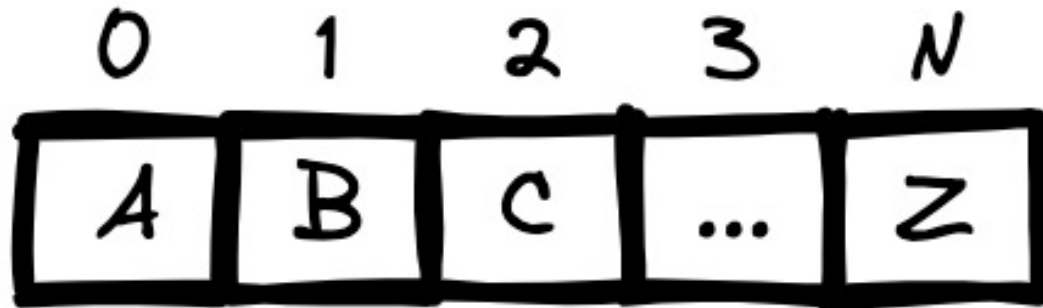


7. 內建物件方法介紹

- Array 物件

Array 物件

- 陣列(array)是資料合集，也是一種物件
- 陣列是以[索引值]來管理操作
- 元素值可重複



Array方法

方法	說明	方法	說明 *為改變元陣列方法
length	陣列長度	slice(i [,end])	擷取第i+1~end的元素
indexOf()	搜尋傳回第一個元素位置	*splice(i [,end] [,ele])	擷取、刪除、新增元素
toString()	轉成字串	forEach()	遍歷陣列內的元素
*push()	陣列末端新增元素	concat()	連接多個陣列建立新陣列
*pop()	刪除陣列末端元素	join()	轉成字串但可自定義
*unshift()	陣列開頭新增元素	*reverse()	陣列元素反轉
*shift()	刪除陣列開頭元素	*sort()	陣列元素排序

🔍 <https://javascript.com/>



7. 內建物件方法介紹

- String 物件

String 物件

●String物件的方法

方法	說明	方法	說明
length	取得字串長度	slice()	從字串中擷取n+1~end字串
trim()	移除字串前後空白	substr()	從字串中擷取n+1~end字串
charAt()	取出第n+1個字	split()	從字串第n+1個字擷取
charCodeAt()	將第n+1個字轉碼	toLowerCase()	轉成小寫
concat()	於字串後串接字串	search()	搜尋字串並取得符合字串
indexOf()	從字串前端搜尋	match()	搜尋字串將符合字串位置
lastIndexOf()	從字串後端搜尋	replace()	搜尋字串傳回第一個符合位置

🔍 <https://javascript.com/>



7. 內建物件方法介紹

- Number 物件

Number 物件

- Number物件提供轉型功能或讀取number屬性

方法	說明	屬性	說明
toString()	轉換字串	NaN	不是數字Not a Number
isNaN()	檢驗是否為NaN	MAX_VALUE()	表示最大值
isInteger()	檢驗是否為整數	MIN_VALUE()	表示最小值
parseInt()	字串轉為整數	NEGATIVE_INFINITY()	負無限大
parseFloata()	字串轉為浮點數	POSITIVE_INFINITY()	正無限大

🔍 <https://javascript.com/>



7. 內建物件方法介紹

- Date 物件

Date 物件

● Date() 處理日期/時間相關的方法

方法	說明	方法	說明
getDate()	取得日(1~31)	setDate()	設定日(1~31)
getFullYear()	取得年(4位)	setFullYear()	設定年(4位)
getMonth()	取得月(0~11)	setMonth()	設定月(0~11)
getDay()	取得星期(0~6)	setDay()	設定星期(0~6)
getTime()	取得自1970/01/01起經過的毫秒	setTime()	設定自1970/01/01起經過的毫秒

🔍 <https://javascript.com/>



7. 內建物件方法介紹

- Math 物件

Math 物件

- Math() 數學相關運算功能的方法
- 不用創建物件，直接使用 Math.PI()

方法	說明	方法	說明
PI()	圓周率	random()	0~1間的亂數
abs()	絕對值	max()	取最大值
ceil()	無條件進位	min()	取最小值
floor()	無條件捨去	pow()	指數運算
round()	四捨五入	sqrt()	平方根

🔍 <https://javascript.com/>



7. 內建物件方法介紹

- RegExp 物件

RegExp物件

- 正規表示式，以字串樣式搜尋或設定字串

- 修飾字

➤ g 整個字串是否有匹配

➤ i 是否區別大小寫

樣式	匹配說明	樣式	說明
[ABC]	ABC任一字元	{n}	n幾次
[^ABC]	ABC以外字元	+	至少一個
A B C	A或B或C	*	0或多個
\w	符合大小寫英文字	?	0或1個
\d	符合數字	^	該行開頭
\s	符合空白字串	\$	該行結尾

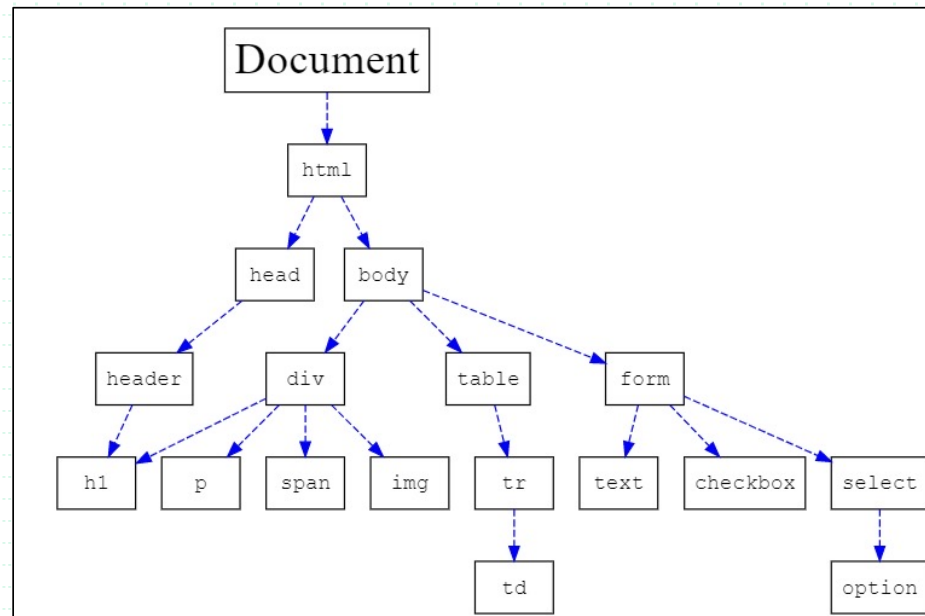
🔍 <https://javascript.com/>



8. HTML DOM物件操作

DOM (Document Object Model)

- DOM是一個以樹狀結構表示HTML文件模型，並定義讓程式可以存取及改變文件架構樣式內容方法



DOM (Document Object Model)

- 這些構成文件的元素 屬性 文字等物件都被稱為節點(Node) , 對應其種類分別有元素節點、屬性節點、文字節點
- DOM就是針對這些節點提供取出、新增、取代、移除等功能的API

取得元素節點

- 以id為取得元素

- 取得元素物件

```
document.getElementById(id)
```

- 以標籤名稱為取得元素

- 回傳HTMLCollection物件

```
document.getElementsByTagName(tag)
```

- 以名稱為取得元素

- 回傳NodeList物件

```
document.getElementsByName(name)
```

- 以class屬性為取得元素

- 回傳HTMLCollection物件

```
document.getElementsByClassName(class)
```

取得元素節點

- 以符合選擇器條件取的元素
 - 以原CSS選擇器語法條件選取
 - `querySelector()`
 - ✓ 傳回元素物件

```
document.querySelector(selector)
```

- `querySelectorAll()`
 - ✓ 傳回NodeList物件

```
document.querySelectorAll(selector)
```


DOM 取得/設定元素文本

- 取得DOM文本

```
element.innerHTML
```

- 設定DOM文本

```
element.innerHTML = text
```

DOM 新增/插入節點

- 新增DOM節點

```
document.createElement(ele)
```

- 插入DOM節點

```
ele.appendChild(node)
```

DOM 設定樣式屬性

- 透過JavaScript設定DOM樣式
 - 此設定方式是行內屬性
 - css屬性有連接字號都要改成駝峰是命名
 - ✓ background-color. ➡ backgroundColor

```
element.style.prop=[ value]
```

🔍 <https://javascript.com/>



9. 事件處理 1

事件簡介

- 瀏覽器顯示網頁有各種事件，例如:點擊按鈕、滑鼠移動等，就是使用者與瀏覽器互動的行為稱為事件行為

滑鼠事件

click
dblclick
contextmenu
mousedown
mouseup
mouseenter
mouseleave
mouseover
mouseout
mousemove

鍵盤事件

keydown
keypress
keyup

表單事件

blur
change
focus
reset
submit

視窗事件

resize
scroll

文件事件

load
ready
unload

事件綁定方式

- 綁定事件三種方式
 - ✓ 標籤內屬性宣告
 - ✓ 元素物件的函數宣告
 - ✓ 使用addEventListenter方法宣告

事件綁定方式 1

- 標籤內屬性宣告
 - **不建議** 此方式處理事件程序

```
<標籤名稱 onevent="JavaScript 函數">
```

事件綁定方式 2

- 元素物件的函數宣告

```
obj.onevent=function(){  
    //javascript code  
}
```


事件綁定方式 3

- 使用addEventListenter方法宣告

```
obj.addEventListener(event, function(){  
    //javascript code  
})
```

事件處理程序

- 事件處理程序三個面項
 - ✓ 哪個元素觸發事件
 - ✓ 觸發什麼事件
 - ✓ 觸發事件的內容或處理程序

onload 事件

- 會等待文件頁面內容及所有圖片載入後才執行程式

```
window.onload=function(){  
    //javascript code  
}
```

🔍 <https://javascript.com/>



10. 事件處理 2

何謂Event物件

●Event 物件

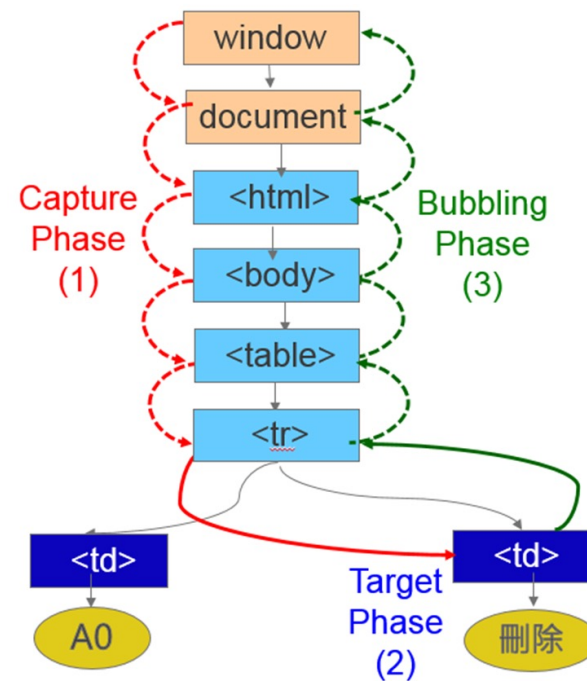
- 網頁上的事件觸發的同時會產生event物件，該物件可取的事件觸發的各種資訊。
- 預設event傳遞參數

event 物件的屬性

- type : 事件的類型
- keyCode : 回傳鍵盤輸入的編碼或滑鼠按鈕的代碼
- pageX , pageY : 滑鼠游標相對於文件左邊界或上邊界的位置
- target : 回傳觸發事件的 DOM 元素
- currentTarget : 回傳事件正在處理時所在的 DOM 元素

event 物件的方法

- `preventDefault()`
 - 停止(取消)預設的動作
- `stopPropagation()`
 - 取消事件氣泡



🔍 <https://javascript.com/>



11. BOM物件

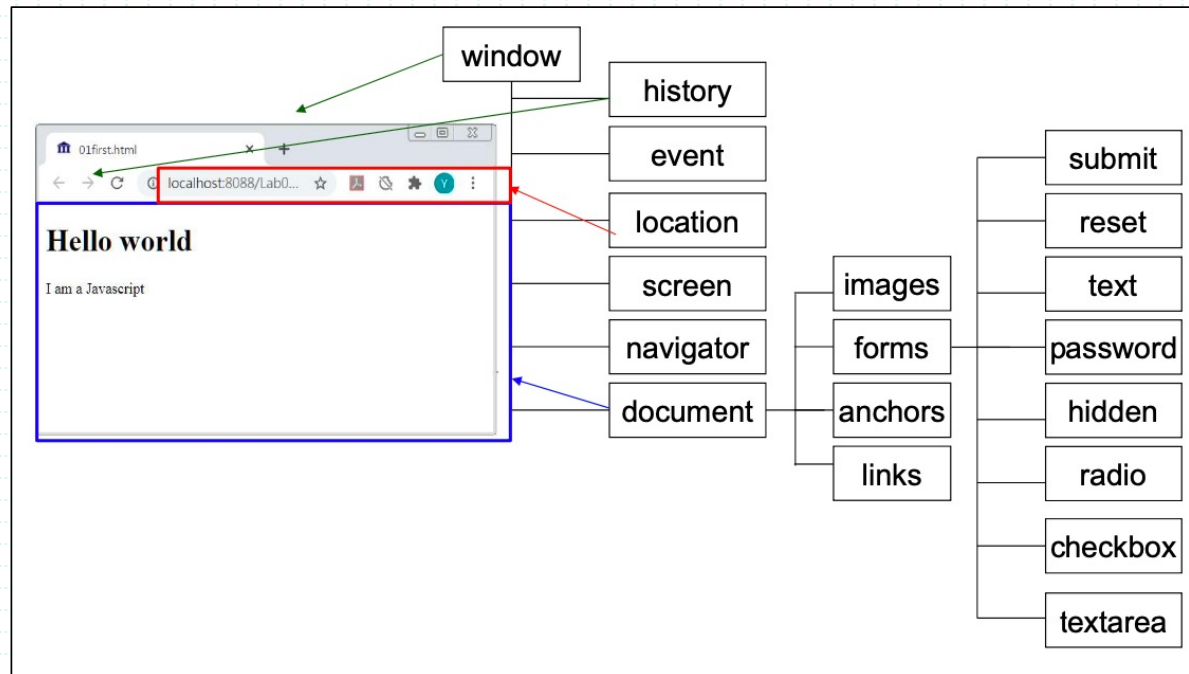
何謂BOM

- 瀏覽器物件
 - 是匯總操作瀏覽器功能物件的總稱。
 - window
 - ✓ 存取瀏覽器物件
 - location
 - ✓ 取得操作顯示頁的位置資訊
 - histroy
 - ✓ 依瀏覽歷史紀錄前後切換頁面
 - navigator
 - ✓ 跨瀏覽器物件

https://javascript.com/

BOM (Browser Object Model)

- BOM是瀏覽器物件模型，是瀏覽器所有功能的核心



JavaScript計時器

- 啟動一個計時器來計算一個程式或操作時間，
每一個計時器必須擁有計時名

方法	說明
<code>console.time(timerName)</code>	開啟一個計時器
<code>console.timeEnd(timerName)</code>	關閉一個計時器

setInterval/setTimeout方法

● setInterval

- 依指定的時間間隔重複執行

```
window.setInterval(function, duration)
```

● setTimeout

- 經過指定時間後，執行單次處理

```
window.setTimeout(function, duration)
```

🔍 <https://javascript.com/>



12. JSON

JavaScript Object Notation

JSON

- JSON即 『JavaScript Object Notation』
- JavaScript 物件表示法
- 屬性名稱必須加雙引號
- 可以分為
 - JSON物件{ }
 - JSON陣列[]

JSON

● JSON物件

```
{  
  "name": "Madame Uppercut",  
  "age": 39,  
  "secretIdentity": "Jane Wilson",  
  "powers": [  
    "Million tonne punch",  
    "Damage resistance",  
    "Superhuman reflexes"  
  ]  
}
```

JSON

● JSON陣列

```
[{  
    "name": "Molecule Man",  
    "age": 29,  
    "secretIdentity": "Dan Jukes",  
    "powers": ["Radiation resistance", "Turning tiny", "Radiat  
ion blast"]  
},  
    {  
    "name": "Madame Uppercut",  
    "age": 39,  
    "secretIdentity": "Jane Wilson",  
    "powers": ["Million tonne punch", "Damage resistance",  
    "Superhuman reflexes"]  
}]
```


JSON

- JavaScript提供兩個方法做物件和JSON資料轉換
 - JSON.parse() : JSON ➡ JS物件
 - JSON.stringify() : JS物件 ➡ JSON

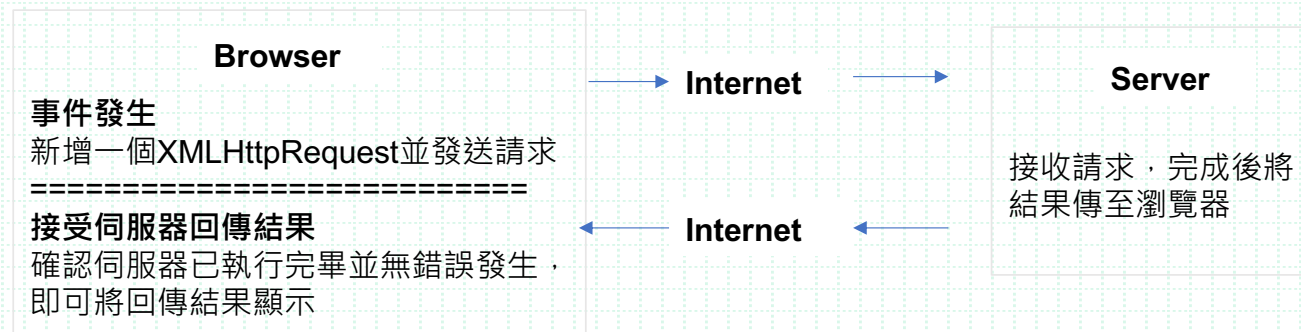
🔍 <https://javascript.com/>



12. AJAX 非同步請求

非同步請求 AJAX

- AJAX即『Asynchronous JavaScript and XML』



Ajax相關API

- XMLHttpRequest()物件來達成此功能
- jQuery有提供包覆的語法
 - \$.ajax()
- JavaScript在ES6發展出Promise物件
 - Fetch API
 - axios API